

Part 2: Interface

1. Objectives

- 1) Students are able to implement interfaces and use them.

2. Instructions

- 1) Create Java Project named “2110215_Exam2_Part2” in your workspace.
- 2) Copy all folders in “toStudent/Part2” to your project directory src folder.
- 3) You are to implement the following classes (details for each class are given in section 4.)
 - a) BaseProduct (package products)
 - b) FreeDiscountProduct (package products)
 - c) PercentDiscountProduct (package products)
- 4) After creating all the required classes, you are required to draw a UML Diagram using ObjectAid. Save it as .png in the root of the project’s folder. (The UML Diagram is worth 2 points.)

3. Problem Statement: Smoke Sales



You work for a grocery store called Smoke. Smoke is about to have its 1-year anniversary celebration, and is planning to do so by giving away discounts for its products in a “Smoke Sale”. Your task as one of the programmers is to work on a discount calculation program to be used at the cash register. The program can be run from the class **Main.java** in the **app** package.

4. Implementation Detail

The class package is summarized below. Note that only important/useful methods are listed.

- private

protected

+ public

Static variables and methods are underlined.

4.1 package discount

****ALREADY GIVEN****

Everything in this package is already written for you. Only important variables and methods will be listed for you to use.

4.1.1. public interface Sellable

a. Variables

None.

b. Methods

Name	Description
+ abstract int <u>calculateDiscount</u> (int quantity)	Method to calculate the total discount value for the given quantity of Sellable objects.

4.1.2. public interface FreeDiscountable

a. Variables

None.

b. Methods

Name	Description
+ abstract int <u>calcFreeDiscountPieces</u> (int totalQuantityBought)	Method to calculate the number of pieces that will get a discount given the total amount of items bought. (Buy X get Y free – given the value of (X+Y), the method returns the value of Y)

4.1.3. public interface PercentDiscountable

a. Variables

None

b. Methods

Name	Description
------	-------------

+ abstract int calcDiscountPerPiece()	Method to calculate the discount value for 1 piece of the product.
--	--

4.2 package products

****PARTIALLY PROVIDED****

This entire package, except for BaseProduct, must be written from scratch.

4.2.1 public class BaseProduct

A base Sellable object in the store with no discounts. This class is almost complete, but is **missing one very important component**. To run the program, you must add that component to this class.

a. Variables

Name	Description
# String productName	The name of the product.
# int price	The default price of the product.

b. Methods

Name	Description
+ BaseProduct(String name, int price)	Initializes the values for the BaseProduct. Make sure to use the appropriate setters.
+ int calculateDiscount(int quantity)	Method from Sellable. Since this product does not offer promotions, the discount is always 0.
+ String toString()	Returns the object as a string.
+ getters and setters for all variables	For setPrice: If the input is less than 0, set price as 0. For setProductName: If the input is a blank string, set the name as "PlaceholderProduct".

4.2.2 public class FreeDiscountProduct extends BaseProduct

A product which can be discounted by the FreeDiscount promotion, and is Sellable.

a. Variables

Name	Description
- int promoQuantity	The promotion quantity of this product type. (The X in Buy X Get Y Free)
- int freeQuantity	The quantity of the product you get for free after buying promoQuantity amount of product. (The Y in Buy X Get Y Free)

b. Methods

Name	Description
+ FreeDiscountProduct(String name, int price, int promoQ, int freeQ)	Initializes the values for the FreeDiscountProduct. Make sure to use the appropriate setters.
+ int calcFreeDiscountPieces(int totalQuantityBought)	Calculates the number of free pieces that will be given at a certain quantity. Use ShopUtil's calculateFreeDiscountPieces for this.
+ int calculateDiscount(int quantity)	Returns the discount. For this product, for the given quantity, the discount is equal to the number of free pieces multiplied by the default price.
+ String toString()	Returns this object as a string. The code for this method is provided in Exam2Part2ProvidedCodes.txt.
+ getters and setters for all variables.	For setPromoQuantity and setFreeQuantity: If the given value is less than 1, set it as 1.

4.2.3 public class PercentDiscountProduct extends BaseProduct

A product which can be discounted with a simple percent discount, and is Sellable.

a. Variables

Name	Description
------	-------------

- double percent	The discount percentage applied to this product.
------------------	--

b. Methods

Name	Description
+ PercentDiscountProduct(String name, int price, double percent)	Initializes the values for the PercentDiscountProduct. Make sure to use the appropriate setters.
+ int calcDiscountPerPiece()	Calculates the discount that will be given per piece. Use ShopUtil's calculateDiscountUsingPercent for this.
+ int calculateDiscount(int quantity)	Returns the discount. For PercentDiscountProduct, the discount is equal to the quantity purchased multiplied by the discount per piece.
+ String toString()	Returns this object as a string. The code for this method is provided in Exam2Part2ProvidedCodes.txt.
+ getter/setter for all variables.	For setPercent: If the input is less than 0, set it as 0. If the input is greater than 100, set it as 100.

4.3 package logic

****ALREADY GIVEN****

Everything in this package is already written for you. Only important variables and methods will be listed for you to use.

4.3.1 public class ShopUtil

a. Variables

None.

b. Methods

Name	Description
<u>+ int calculateFreeDiscountPieces(int freeQ, int promoQ, int totalQ)</u>	Static method. Calculates the number of free pieces of product given in a Buy X Get Y Free. Put the X in freeQ, Y in promoQ, and the total purchase amount in totalQ.

+ int <u>calculateDiscountUsingPercent</u> (int <u>price</u> , double <u>percent</u>)	Static method. Calculates the discount, given the original price and the percentage.
--	--

4.3.2. public class BillItem

This class will be referred to in the method you will have to write in Main. This class represents a single line in a bill or receipt.

a. Variables

Name	Description
Sellable itemInLine	The product in this line in the bill.
int quantity	The amount of the product purchased in the bill.

b. Methods

Name	Description
+ int getPrice (boolean withDiscount)	Returns the price of this line in the bill. If withDiscount is true, all available discounts on the product will be applied.
+ getter/setter for all variables	

4.3.3. public class Bill

This class will be referred to in the method you will have to write in Main. This class represents a bill you will receive when shopping at the store.

a. Variables

Name	Description
+ ArrayList<BillItem> itemsInBill	The list of BillItem lines in the bill. Each BillItem is one line in this bill.

b. Methods

None.

4.4 package app

****PARTIALLY PROVIDED****

You need to complete the addItemToBill method inside Main for the program to run properly.

4.4.1. public class Main

a. Variables

Name	Description
<u>Bill bill</u>	The bill containing all the items you are about to check out.
<u>ArrayList<Sellable> shop</u>	The quantity of the product you get for free after buying promoQuantity amount of product. (The Y in Buy X Get Y Free)
<u>Scanner sc</u>	A scanner that receives input from the user.

b. Methods

Name	Description
<u>+ addItemToBill()</u>	The description of how to write this method is provided within the Main file. You will need to write your codes inside the try brackets, divided into two parts: Before the “Choose Quantity” line: A1. Accept a string input with sc.nextLine() A2. Attempt to parse the string into an int (the try/catch will handle bad cases automatically) A3. Use the int from step A2 to get a Sellable object from the ArrayList called shop After the “Choose Quantity” line: B1. Accept a string input with sc.nextLine() B2. Attempt to parse the string into an int (the try/catch will handle bad cases automatically) B3. Use the Sellable from step A3 and the quantity from B2 to initialize a new BillItem B4. Add the BillItem in step B3 to bill.itemsInBill NOTE: There is no JUnit available for this function. If you can correctly add items to the bill in the application, then your function is correct.

5. Score Criteria (Marks will be scaled down to 10)

Your work will be checked against JUnit test cases, as well as program functionality.

Scorable items are shown in the list below.

- 1) BaseProductTest
 - a) sellableTest (3 marks)
- 2) FreeDiscountProductTest
 - a) constructorTest (1 mark)
 - b) setPromoQuantityTest (1 mark)
 - c) setPromoQuantityLessThanOneTest (1 mark)
 - d) setFreeQuantityTest (1 mark)
 - e) setFreeQuantityLessThanOneTest (1 mark)
 - f) calculateDiscountTest (1 mark)
 - g) freeDiscountableTest (2 marks)
 - h) notPercentDiscountableTest (2 marks)
 - i) toStringTest (0.5 mark)
- 3) PercentDiscountProductTest
 - a) constructorTest (1 mark)
 - b) setPercentTest (1 mark)
 - c) setPercentLessThanZeroTest (1 mark)
 - d) setPercentGreaterThanHundredTest(1 mark)
 - e) calculateDiscountTest (1 mark)
 - f) percentDiscountableTest (2 marks)
 - g) notFreeDiscountableTest (2 marks)
 - h) toStringTest (0.5 mark)
- 4) Main Functionality
 - a) The method addItemToBill must be completed and products is able to be added to Bill in the program. (5 marks)
- 5) UML Diagram
 - a) A correct UML Diagram must be put in the project directory. Please include all classes inside the packages discount and products. (Other classes are optional.) (2 marks)